Personal Finance Advisor Bot - Project Documentation

Program: AI Specialist BFSI (Skill Wallet) **Author:** Ketan Shinde **Repository:** https://github.com/ketanshind0987-ui/personal-finance-advisor-bot

1. Introduction

The Personal Finance Advisor Bot is an AI-powered financial planning assistant that helps individuals take control of their personal finances. Users can record monthly income, log expenses by category, receive personalized budget and savings recommendations, and track their progress from a single dashboard.

2. Problem Statement

Many people, especially students and early-career earners, track spending manually or not at all. This makes it hard to see where money goes, stay within a budget, or reach a savings goal. Manual tracking is slow and error-prone, and generic budgeting advice is rarely tailored to the individual's income and spending pattern.

3. Objectives

- Provide a simple, secure place to record income, expenses and savings goals.
- Summarize spending by category so users can see where their money goes.
- Generate personalized budget and savings recommendations using AI.
- Keep the system easy to run locally and easy to extend.

4. Features

- User registration and login with secure password hashing and session management
- Monthly income and savings-goal management
- Expense tracking by category, with expense deletion
- Dashboard showing total income, total expenses and remaining balance
- Category-wise spending summary
- Budget and savings recommendations
- Optional Gemini AI integration, with built-in local recommendations when no API key is set

- JSON summary API
- Responsive dark user interface

5. Technology Stack

Layer	Technology
Language	Python
Backend framework	Flask
ORM	Flask-SQLAlchemy
Database	SQLite
Templating	Jinja2
Frontend	HTML, CSS, JavaScript
AI engine	Google Gemini (optional)

6. System Architecture

The application follows a three-layer full-stack design:

1. **Presentation layer:** HTML templates (Jinja2) with CSS and JavaScript, served from the `templates` and `static` folders.
2. **Application layer:** Flask routes in `app.py` handle authentication, income and expense modules, the dashboard and the AI advice endpoint.
3. **Data layer:** SQLAlchemy models persist user accounts, monthly income and savings goals, and expense records in an SQLite database stored in the `instance` folder.

Request flow: the user interacts with the web page, the page sends a request to a Flask route, the route reads or writes the database, and the result is rendered back as HTML or returned as JSON.

AI flow: when the user clicks "Get AI Advice", the `/api/ai-advice` route gathers the user's income, savings goal and category-wise expenses, and sends them to Gemini for recommendations. If no Gemini key is configured, the app falls back to local rule-based recommendations.

7. Project Structure

Item	Purpose
`app.py`	Main Flask application: routes, models, AI logic
`templates/`	HTML pages (Jinja2)
`static/`	CSS, JavaScript and other static assets
`instance/`	SQLite database
`requirements.txt`	Python dependencies
`README.md`	Quick-start guide

8. Work Completed by Epic

Epic	What was done
Epic 1: Model Selection and Architecture	Chose Flask with SQLAlchemy and SQLite for the backend, and Gemini as the optional AI engine. Defined the three-layer architecture above.
Epic 2: Core Functionalities Development	Built authentication, income and savings-goal management, expense tracking, and the spending summary.
Epic 3: App.py Development	Implemented the Flask application, database models, dashboard route, the `/api/ai-advice` endpoint and the JSON summary API.
Epic 4: Frontend Development	Created the responsive dark UI with registration, login, dashboard and expense pages.

[Add any further epics from your Skill Wallet workspace here.]

9. Setup and Installation

1. Create a virtual environment: `python -m venv venv`, then activate it (`venv\Scripts\activate` on Windows, `source venv/bin/activate` on Linux/macOS).
2. Install packages: `pip install -r requirements.txt`

3. Copy `.env.example` to `.env` and set a strong `SECRET_KEY`. For Gemini AI, add `GEMINI_API_KEY=your_key_here`.
4. Start the app: `python app.py`
5. Open http://127.0.0.1:5000 in a browser.

10. Demo Walkthrough

1. Register a new user.
2. Set monthly income and savings goal.
3. Add expenses such as rent, food, transport and entertainment.
4. Review the dashboard totals.
5. Click "Get AI Advice".
6. Review the category-wise spending and savings recommendations.

11. Screenshots

[Insert screenshot: Registration / Login page]

[Insert screenshot: Dashboard with income, expenses and balance]

[Insert screenshot: Add expense page]

[Insert screenshot: AI advice output]

12. Testing

The application was tested manually by running the full workflow: registering, logging in, setting income and goals, adding and deleting expenses, viewing the dashboard, and requesting AI advice with and without a Gemini key. Automated unit tests can be added as a future improvement.

13. Limitations

- Intended as an educational project, not professional financial advice.
- Uses SQLite, which suits a single-user or small deployment.
- AI recommendations depend on the quality of the data the user enters.

14. Future Enhancements

- Predictive spending analytics

- Investment suggestions
- Goal-based savings tracking
- Advanced AI-driven financial health monitoring
- Charts and downloadable monthly reports
- Automated test suite and production deployment

15. Conclusion

The Personal Finance Advisor Bot reduces the effort of manual financial tracking by combining expense management, a clear dashboard and AI-driven budget recommendations in one lightweight application. Its modular Flask design makes it easy to extend with new analytics and planning features.

Note: This project is for educational purposes. Do not enter sensitive banking credentials or card numbers.